PRODUCT REQUIREMENTS DOCUMENT

Adversarial Game-Playing Agent

Connect Four AI

Interactive gameplay, adversarial search, and algorithm benchmarking

Field	Specification
Project type	AI/ML + Full-Stack Web Application
Version / Status	1.0 / In Development
Frontend	React.js, Vite, JavaScript
Backend	Python, FastAPI
AI methods	Random, Greedy, Minimax, Alpha-Beta, optimized search, MCTS/UCT
Target users	Students, AI/ML learners, strategy-game enthusiasts, developers

Purpose: define scope, requirements, architecture, API contracts, testing, team responsibilities, and release criteria.

1. Executive Summary

The Adversarial Game-Playing Agent is a browser-based application that lets users play Connect Four against AI agents using different decision-making algorithms. Connect Four uses a 7-column × 6-row board; players alternate dropping discs, and the first to connect four horizontally, vertically, or diagonally wins.

The product combines a graphical game interface, configurable AI opponents, backend game-state validation, and performance analytics. It is both a playable game and an educational platform for comparing adversarial search methods.

- Human vs. AI gameplay; Human vs. Human and AI vs. AI may be supported as scope allows.
- Random and Greedy baseline agents.
- Minimax with depth limits; Alpha-Beta pruning; optimized search with move ordering and transposition tables.
- Monte Carlo Tree Search (MCTS) using Upper Confidence Bound for Trees (UCT).
- Win/draw/loss, decision time, node counts, pruning statistics, and algorithm-specific metrics.

2. Problem Statement

Traditional Connect Four games emphasize gameplay and often do not explain AI move selection or compare search strategies. Separate implementations also make it difficult to visualize behavior and measure computational efficiency under consistent conditions. This project combines interactive gameplay, a modular AI engine, and a performance-analysis interface.

3. Product Vision

Build a reliable, interactive educational platform that demonstrates adversarial game-playing techniques through Connect Four.

- Engaging game interface and modular AI-agent architecture.
- Validated backend API and transparent algorithm comparisons.
- Quantitative, reproducible evaluation and a deployable browser application.

4. Goals and Scope

4.1 Primary goals

- Provide a fully playable game with correct rules and win/draw detection.
- Integrate AI algorithms through a common interface.
- Allow opponent and difficulty selection.
- Measure decision time/search effort and display comparisons.
- Keep the application maintainable, testable, and deployable.

4.2 Educational goals

- Demonstrate game-tree search and Minimax.
- Show Alpha-Beta pruning and the effect of move ordering/transposition tables.
- Introduce MCTS/UCT and compare deterministic search with simulation-based search.

4.3 Initial out of scope

- Online multiplayer matchmaking/chat, profiles/social features, native mobile app.
- Reinforcement learning/neural-network training, tournament infrastructure, distributed AI computation. These may be future enhancements.

5. Target Users

User group	Needs
Casual players	Responsive gameplay, clear controls, adjustable challenge.
Students and learners	Algorithm explanations, move statistics, search metrics, visualizations.
Developers/researchers	Reproducible experiments, consistent conditions, detailed metrics, exports.

6. Functional Requirements

FR-01 — Game board

- Render 7 columns × 6 rows; distinguish players; highlight selected/hovered columns; animate drops; prevent moves into full columns; show turn/status; highlight winning four; disable moves after game ends.

FR-02 — Game-state management

- Track board, current player, legal columns, move history/count, status, winner, last move, and winning sequence. Backend validation is authoritative.

FR-03 — Game modes

- Human vs. AI is core. Human vs. Human can run locally. AI vs. AI is useful for demonstrations and benchmarking and may follow the primary gameplay flow.

FR-04 — AI selection

- Offer Random, Greedy, Minimax, Alpha-Beta, optimized Alpha-Beta, and MCTS/UCT through a common selector. Show agent name and supported configuration.

FR-05 — Difficulty

- Configure Minimax depth and MCTS simulation/iteration budgets. Illustrative tiers: Easy (Random/shallow), Medium (Greedy/moderate), Hard (deeper Alpha-Beta), Expert (optimized Alpha-Beta or configured MCTS). Establish values through tests.

FR-06 — Legal moves

- Reject invalid columns, full columns, moves after terminal states, wrong-player moves, and malformed requests without corrupting state.

FR-07 — Win/draw

- Detect horizontal, vertical, and both diagonal wins. Declare a draw when the board is full and no winner exists.

FR-08 — Controls

- New Game and Restart. Undo, agent/difficulty changes, and pause/resume for AI-vs-AI may be added where supported. Reset all per-game state and metrics on a new game.

FR-09 — AI visualization

- Show selected column, depth, nodes evaluated, Alpha-Beta cutoffs, decision time, candidate scores, MCTS iterations and visit counts where available. Metrics are algorithm-specific.

FR-10 — Dashboard

- Show W/D/L, win rate, average/median/maximum decision time, nodes, search depth, pruning, MCTS metrics, and memory only where measured reliably. Mark inapplicable values as unavailable.

FR-11 — Benchmarking

- Run selected agents in automated matches; alternate starting players; use reproducible seeds where supported; record outcomes and metrics; aggregate and chart results; export CSV/JSON.

FR-12 — Error handling

- Handle backend failures, AI timeouts, invalid moves/configuration, malformed responses, and unexpected failures. Provide actionable errors and safe retry/restart.

7. Non-Functional Requirements

ID	Requirement
Performance	Keep interaction responsive; bound expensive searches; separate AI compute time from network/rendering time; set numeric latency goals after profiling.
Correctness	Maintain valid states, enforce rules, stop at terminal states, return legal AI moves, and calculate metrics consistently.
Reliability	Errors must not corrupt state; API schemas must be documented; restart and recovery must work.
Security	Validate requests, limit benchmark workloads, configure production CORS, protect credentials, and reject executable payloads.
Maintainability	Separate game engine/API/AI; use shared agent interface; modular UI; automated tests and setup documentation.
Usability/accessibility	Responsive layout, readable contrast, keyboard interaction where practical, accessible labels, and status not conveyed by color alone.

8. Technical Architecture

Layer	Responsibilities
Frontend	React.js, Vite, JavaScript, CSS/styling library; board, controls, statistics, charts.
Backend	Python, FastAPI, Pydantic validation, game-state validation, AI integration, benchmark orchestration.
AI engine	Shared state, legal moves, terminal detection, heuristic evaluation, baseline agents, Minimax, Alpha-Beta, optimized search, MCTS/UCT.

Request flow: frontend calls FastAPI; backend validates and invokes a game/AI operation; backend returns structured data; frontend updates board and metrics.

8.1 Proposed repository structure

```
Adversarial-Game-Playing-Agent-/
■■■ backend/
■   ■■■ main.py
■   ■■■ requirements.txt
■   ■■■ app/
■   ■   ■■■ api/ (game_routes.py, ai_routes.py, benchmark_routes.py)
■   ■   ■■■ core/ (game_engine.py, game_state.py, validation.py)
■   ■   ■■■ agents/ (base_agent.py, random_agent.py, greedy_agent.py,
■   ■   ■           minimax_agent.py, alphabeta_agent.py,
■   ■   ■           optimized_agent.py, mcts_agent.py)
■   ■   ■■■ services/ (game_service.py, benchmark_service.py)
■   ■   ■■■ schemas/ (game_schemas.py, benchmark_schemas.py)
■   ■■■ tests/
■■■ frontend/
■   ■■■ src/
■       ■■■ components/ (GameBoard, GameCell, GameControls,
■       ■               AgentSelector, GameStatus, MetricsPanel, BenchmarkCharts)
■       ■■■ pages/ (HomePage, GamePage, AnalyticsPage)
■       ■■■ services/api.js
■       ■■■ hooks/useGame.js
■       ■■■ App.jsx
■       ■■■ main.jsx
■■■ docs/ (PRD.md, API_SPECIFICATION.md)
■■■ .gitignore
■■■ README.md
```

This is a proposed target structure, not an instruction to replace the current repository. Add files incrementally.

9. Game-State Representation

Use a consistent six-row by seven-column matrix: 0 = empty, 1 = Player 1, 2 = Player 2.

- Maintain/derive current player, legal columns, move count, terminal status, winner, and winning coordinates.
- Use identical coordinate conventions in game engine and agents.
- Frontend may keep a display copy, but backend validates every action.

10. AI Algorithm Requirements

10.1 Random Agent

- Choose randomly from legal moves; support seeded randomness; report move and decision time.

10.2 Greedy Agent

- Score candidate moves locally; prioritize immediate wins and block immediate threats where possible; evaluate board patterns; document its limits on long-term planning.

10.3 Minimax

- Explore alternating maximizing/minimizing turns; use terminal outcomes and heuristic values at depth limit; support configurable depth.

10.4 Alpha-Beta

- Maintain alpha/beta bounds and prune branches that cannot change the result. For the same horizon/evaluation, preserve Minimax's optimal value; tie-breaking may differ. Report node/pruning metrics.

10.5 Optimized Alpha-Beta

- Consider center-first move ordering, immediate-win detection, transposition table, board hashing, iterative deepening, efficient win detection, and tested heuristics. Benchmark optimizations separately where practical.

10.6 MCTS with UCT

- Implement selection, expansion, rollout/simulation, and backpropagation; configure iteration limits; use correct player perspective for values; report iterations and timing.

11. Heuristic Evaluation

Evaluate nonterminal boards using immediate wins, opponent threats, two/three-in-a-row patterns, open-ended connections, center control, blocking opportunities, and threats that can be prevented. Terminal scores must dominate ordinary heuristic scores. Tune weights empirically.

12. API Requirements

These proposed route names and examples must be finalized before integration. Use consistent JSON schemas.

12.1 Health — GET /api/health

```
{ "status": "ok" }
```

12.2 Create game — POST /api/game/new

```
Request:
{
  "mode": "human_vs_ai",
  "ai_agent": "alphabeta",
  "difficulty": "medium",
  "starting_player": 1
}

Example response:
{
  "game_id": "example-game-id",
  "board": [],
  "current_player": 1,
  "status": "active",
  "legal_moves": [0, 1, 2, 3, 4, 5, 6]
}
```

The empty board array is illustrative; the actual response must contain the full 6×7 board.

12.3 Human move — POST /api/game/{game_id}/move

```
Request: { "column": 3 }

Example response:
{
  "board": [],
  "current_player": 2,
  "status": "active",
  "winner": null,
  "last_move": {"player": 1, "column": 3, "row": 5}
}
```

12.4 AI move — POST /api/game/{game_id}/ai-move

```
{
  "selected_column": 2,
  "board": [],
  "status": "active",
  "winner": null,
  "metrics": {
    "decision_time_ms": 12.5,
    "nodes_evaluated": 1000,
    "pruned_nodes": 250
  }
}
```

Example metric values are illustrative only; return only metrics actually measured for the selected agent.

12.5 Retrieve state — GET /api/game/{game_id}

Return board, current player, legal moves, status, winner, and move history.

12.6 Agent metadata — GET /api/agents

Return agent identifiers, names, descriptions, supported parameters, and difficulty options.

12.7 Start benchmark — POST /api/benchmark/run

```
{
  "agents": ["random", "minimax", "alphabeta"],
  "games_per_pair": 20,
  "search_depth": 4,
  "seed": 42
}
```

Validate workload limits. Return results or a job identifier if execution is asynchronous.

12.8 Retrieve benchmark — GET /api/benchmark/{benchmark_id}

Return match outcomes, aggregate statistics, configuration, and per-agent metrics.

12.9 Error format

```
{
  "error": {
    "code": "INVALID_MOVE",
    "message": "The selected column is full."
  }
}
```

Distinguish invalid actions from internal errors and AI timeouts.

13. Frontend Requirements

13.1 Home page

- Project name and description; Play Game and Algorithm Comparison actions; How It Works and short algorithm introductions.

13.2 Game page

- Board, turn/status, AI selector, difficulty/search configuration, New Game/Restart, metrics, loading/error states.
- Prevent duplicate requests while AI is calculating.

13.3 Algorithm details

- Show algorithm, explanation, configuration, timing, and applicable metrics.
- Show concrete measurements and move scores where implemented; do not claim to expose a complete internal reasoning trace.

13.4 Analytics page

- W/D/L, win-rate, time, node, pruning and MCTS comparisons; benchmark configuration; CSV/JSON export.
- Label units, sample size, and experimental conditions.

13.5 Visual design

Use a polished technical aesthetic: dark navy/charcoal background, blue/cyan accents, contrasting discs, clear typography, subtle animation, consistent spacing, rounded panels, and responsive sizing. Keep the board central during gameplay.

14. Backend Requirements

- Expose FastAPI and validate payloads with Pydantic.
- Centralize game-state updates and rules; document endpoints and use a common agent interface.
- Return structured metrics; handle invalid requests, timeouts, and compute limits.
- Support benchmarking and a health endpoint; configure CORS for the production frontend origin.
- In-memory storage is acceptable for the MVP if persistence is not required; document that restarts may clear games.

15. Database Requirements

Database is optional for the first playable release. Add PostgreSQL if persistent game history or benchmark storage is needed.

Table	Potential fields
Games	Game ID, creation timestamp, mode, agent, configuration, status, winner.
Moves	Move ID, game ID, player, column, row, timestamp, decision time.
Benchmarks	Benchmark ID, configuration, creation timestamp, completion status, aggregate metrics.
Benchmark Results	Benchmark ID, agent names, outcomes, search statistics, timing statistics.

16. Testing Strategy

16.1 Unit tests

- Board initialization, legal moves, gravity, full-column rejection.
- Horizontal/vertical/both diagonal wins, draw detection, terminal-state handling.
- AI legal moves, heuristic evaluation, search bounds, and pruning.

16.2 API tests

- Health, game creation, valid/invalid/full-column moves, AI requests.
- Invalid IDs/configurations, terminal-state moves, timeout and error handling.

16.3 Frontend tests

- Board rendering, column selection, status updates, agent selector, loading/errors, restart, responsive layout.

16.4 Integration tests

- Frontend/backend communication and state consistency; AI legality and terminal outcomes; metric and benchmark rendering.

16.5 AI evaluation

- Play against Random baseline; alternate first player; keep rules/configuration consistent.
- Record outcomes, decision time, and applicable metrics; repeat experiments and report sample sizes.

17. Acceptance Criteria

- User can start a game; board is 6×7.
- Valid moves update correctly; invalid moves are rejected.
- Horizontal, vertical, diagonal wins and draws are detected.
- Users can select supported agents and AI returns legal moves.
- Frontend/backend communication is reliable and errors are handled.
- Applicable metrics are accurate; benchmarks are reproducible and documented.
- Application works in a supported desktop browser.
- README documents setup, configuration, testing, and execution.
- Deployed frontend communicates with deployed backend.

MCTS and advanced optimizations can be delivered incrementally if the release scope clearly identifies what is included.

18. Development Roadmap

Phase	Work	Deliverable
1. Foundation	Verify environments; agree schemas/state; implement health endpoint.	Running frontend/backend and integration contract.
2. Game engine	Board, legal moves, move application, win/draw detection, unit tests.	Reliable game engine.
3. Frontend	Board UI, turn indicators, API integration, restart, loading/errors.	Playable interface.
4. Baseline AI	Random/Greedy agents, selector, decision time, legality tests.	Human vs. AI baseline.
5. Search	Minimax, Alpha-Beta, depth, ordering, transposition table, MCTS/UCT.	Interchangeable agents.
6. Analytics	Automated matches, metrics, charts, CSV/JSON export.	Comparison dashboard.
7. Release	Integration tests, profiling, deployment, production checks, docs.	Tested deployable application.

19. Team Responsibilities

Part A — Frontend and Backend Integration (you)

- Maintain repository structure and environments.
- Build React UI, game engine, API contracts, and game-state validation.
- Integrate frontend/backend and teammate's agents via shared interface.
- Display metrics/results; write integration tests; manage deployment/documentation.

Part B — AI Algorithms (teammate)

- Implement Random/Greedy, Minimax, Alpha-Beta, optimized search, and MCTS/UCT.
- Design/test heuristic evaluation; collect statistics; compare performance.
- Document agent parameters, assumptions, and limitations.

Shared responsibilities

Agree on board representation, player IDs, legal-move conventions, terminal semantics, agent interface, API schemas, metric definitions, tests, and benchmark methodology before integration.

20. Deployment Requirements

- Host React frontend and FastAPI backend separately or through a documented deployment setup.
- Configure production API URL, CORS, environment variables, logging, workload limits, and health checks.
- Document deployment; avoid local development-path dependencies.
- Deploy a database only if persistence is in scope.

21. Risks and Mitigations

Risk	Mitigation
Slow AI decisions	Depth/iteration limits, profiling, timeouts.
Illegal AI moves	Shared legal-move generation and unit tests.
Contributor interface mismatch	Agree/test shared contract early.
Misleading benchmarks	Alternate first player; document conditions.
Inconsistent state	Centralize updates and validation.
Misread MCTS metrics	Document value perspective and metric definitions.
Benchmarks block gameplay	Workload limits; asynchronous jobs if needed.
Production API/CORS issues	Test production URLs, origins, and health checks.

22. Future Enhancements

- Multiplayer, profiles, persistent history, tournaments, downloadable replays.
- Step-by-step move and search-tree visualizations.
- Configurable heuristic weights, reinforcement-learning agents, neural-network evaluation.
- Opening-position benchmarks and AI-vs-AI tournaments.

Prioritize future features only after the core game and algorithm comparisons are reliable.

23. Definition of Success

The project succeeds when users can play Connect Four against multiple AI agents, inspect meaningful algorithm-specific metrics, and compare performance under documented conditions. The result must be reliable, modular, testable, and deployable.

Core principle: Correct game rules first, stable API integration second, AI algorithm integration third, and meaningful benchmarking and visualization fourth.